

Nimthara - Bilingual Security Video Guide

Sinhala action guide + English speech + code segments + test evidence

Member	R.M.K.N. Gunasena (Nimthara)
Original website	http://localhost:5174/login
Secure website	http://localhost:5173/login
Assigned scope	V-13, V-14, V-16, V-17 and V-20 - browser safety, upload validation and response confidentiality

Common Sinhala guide

මුලින් මේ පොදු setup එක බලලා browser එක සූදානම් කරන්න

Demo setup සහ record කරන පොදු ක්‍රියා

- Original website: http://localhost:5174/login.
- Secure website: http://localhost:5173/login.
- Username: cashier.colombo. Local demo password එක login field එකට type කරන්න; password එක video එකේ නොපෙන්වන්න.
- Browser tabs දකෙක් නියාගෙන Ctrl + Tab භාවිතා කර Original සහ Secure tabs අතර මාරු වන්න.
- හැම action එකකටම cursor එක field/button/result එක උඩට ගෙන තත්පර දකෙක් නවත්වලා screenshot ගන්න.
- මුලින් Original result එක පෙන්වන්න; පස්සේ ඒ bug එක කියන්න; ඊළඟට Secure result, code segment සහ test result පෙන්වන්න.
- Live database එකට අනවශ්‍ය destructive action කරන්න එපා. Test database, code සහ existing evidence භාවිතා කරන්න.

Video introduction එකේ කියන්න

මමෙ video එකේදී මම Pubudu Tire Management System එකේ මට assign කරලා තිබෙන security workstream එක පෙන්වනවා. හැම vulnerability එකකටම මුලින් Original version එකේ අවදානම පෙන්වනවා. ඊට පස්සේ Secure version එකේ කළ fix එක, අදාළ code segment එක සහ automated test evidence එක පෙන්වනවා. Password, token, secret හෝ personal account information screen එකේ පෙන්වන්නේ නැහැ.

English introduction - say this

"Hello. In this section I will demonstrate my assigned security workstream in the Pubudu Tire Management System. For each vulnerability, I will first show the Original behavior, explain the security impact, and then show the Secure behavior, the relevant code change, and the automated test evidence. Passwords, tokens, secrets and personal account information are kept hidden."

V-13 - Sinhala guide first, English speech underneath

Sinhala guide - මමේ කතෘවස කරන විදිහ

Video steps

Original print/report flow එකමේ stored HTML payload එක code/evidence වලින් පෙන්වන්න; live external token theft කරන්න එපා. Secure escapeHtml.js, print call sites සහ test result පෙන්වන්න. Screenshot names: V13-01-original-unsafe-print.png, V13-02-secure-escaped-output.png, V13-03-escape-code-test.png.

V-13 - Stored XSS කියන්නමේ මොකක්ද?

XSS කියන්නමේ user-controlled text browser එක HTML/JavaScript ලෙස interpret කරන අවස්ථාව. React JSX සාමාන්‍යයෙන් text escape කරනවා. නමුත් original app එක receipt සහ report HTML template string වලින් හදා document.write() කරනවා. ඒ තැන Reactගේ protection එක භාවිතා නොවෙයි. Customer name, product name, invoice note වැනි strings raw HTML තුළ interpolate වුණා. Attacker දත්තයක් save කළාම එය database එකමේ තැන්පත් වලො පසුව admin invoice print කළ විට ක්‍රියාත්මක වුණා - එනිසා stored XSS.

Attack chain එක

```
Cashier/customer-controlled name ■■■ note ■■■ HTML payload ■■■■  
→ database ■■■ ■■■■■■■■ ■■  
→ admin Sales History ■■■ invoice print ■■■■  
→ print window HTML parser payload ■■ tag/script ■■■ ■■■  
→ original localStorage token ■■ script ■■■ ■■■■■■ ■■■■ ■■■■
```

V-13, V-16 සහ V-06 එකට සම්බන්ධ වුණා: XSS + readable browser token + expiry නැති token. Demo එකමේ malicious payload live external address එකකට යැවීම අවශ්‍ය නැහැ; test එකමේ HTML parser img element නොහදන බව පෙන්වන්න.

Fix එක සහ code segment

src/lib/escapeHtml.js හි escapeHtml() අනුප්‍ර පහ escape කරයි: &, <, >, ", ' . & මුලින් replace කරන්නමේ පසුව සෑදෙන HTML entities double-escape නොවීමට. html tagged template helper interpolation හැම එකක්ම escape කරයි; raw() භාවිතා කරන්නමේ application හදන trusted markup පමණයි. escapeJsString() වනෙම context සඳහා; HTML escaping එක JavaScript string context එකට හැම විටම අරමාණවත් නැහැ.

```
return String(value)  
  .replace(/&/g, "&amp;")  
  .replace(/</g, "&lt;")  
  .replace(/>/g, "&gt;")  
  .replace(/"/g, "&quot;")  
  .replace(/'/g, "&#39;");
```

මෙය src/lib/escapeHtml.js හි logic එක. text එක බවට හැරුණාම browser එක image element එකක් නොහදයි. Receipt/report call sites: src/pages/Invoice/CreateInvoice.jsx, src/pages/Invoice/SalesHistory.jsx, src/components/sales-report/PdfGenerator.jsx,

src/pages/Report/ProfitReport.jsx. src/lib/escapeHtml.test.js හි XSS payload, quotes/attribute breakout, </script> සහ සාමාන්‍ය customer names පරීක්ෂා කරයි. CSP එක තවත් protection layer එකක්; escaping වෙනුවට CSP පමණක් ආරම්භකවත් නැත.

English speech - read this in the video

V-13 - Stored cross-site scripting

[Show the Original print or report flow and the original unescaped template code.]

"The Original application inserted customer and product values into HTML strings and opened them with document.write(). React escaping was not applied in that print path. A malicious customer name or product name could therefore be stored in the database and execute later when an administrator printed an invoice. This is stored XSS."

[Show the Secure escape helper, call sites and test.]

"The Secure version escapes HTML context characters before interpolation. The safe template helper escapes every value by default, while raw markup is limited to trusted application-generated markup. The regression tests verify image payloads, quote and attribute breakout, closing script tags and normal names. The payload is rendered as text and is not parsed as an element."

Key code segments and file locations

Escape HTML context characters

File: demo/secure/pubudu-pos-front-end-secure/src/lib/escapeHtml.js Lines: 54-63

```
return String(value)
  .replace(/&/g, '&amp;')
  .replace(/</g, '&lt;')
  .replace(/>/g, '&gt;')
  .replace(/"/g, '&quot;')
  .replace(/'/g, '&#39;');
```

Security lint blocks unsafe sinks

File: demo/secure/pubudu-pos-front-end-secure/eslint.security.config.js Lines: 55-76

```
// document.write, innerHTML and dangerouslySetInnerHTML
// are security-lint violations unless justified.
```

V-14 - Sinhala guide first, English speech underneath

Sinhala guide - මේ කොටස කරන විදිහ

Video steps

Original Git history එකේ .env tracked බව පෙන්වන්න. Secure .gitignore, .env.example සහ CI check පෙන්වන්න. Actual secret value එක screen එකේ පෙන්වන්න එපා. Screenshot names:

V14-01-original-env-history.png, V14-02-secure-ignore-rule.png,
V14-03-secret-scan-test.png.

V-14 - .env file එක Gitට යාම

Frontend .env file එක version control එකේ තිබුණා. .env හි hostnames/config පමණක් තිබුණත් environment files commit කිරීම වැරදි default එකක්; අනාගත secret එකක් අහමිලෙන් පළ විය හැක. Current tree එකෙන් .env ඉවත් කර .gitignore rule හා .env.example දන්නා. CI එක tracked .env තිබේද සහ ignore rule තිබේද බලයි. පරණ Git history එක තව තියනවා; history rewrite/host rotation NF-5 ලෙස ඉතිරි. “අපි historical leak සම්පූර්ණයෙන් මකා දැමීම” කියන්න එපා.

English speech - read this in the video

V-14 - Environment file committed to source control

[Show the Original Git evidence for the .env file.]

“The Original frontend had an environment file tracked in version control. Even when the current file contains configuration rather than a secret, committing environment files creates a direct path for future credentials and deployment details to be exposed.”

[Show the Secure .gitignore, .env.example and CI secret check.]

“The Secure version removes the real environment file from the working tree, adds an explicit environment-file ignore rule, and keeps only a safe example template. CI checks both that no environment file is tracked and that the ignore rule remains present. Historical repository exposure must still be handled by rotating affected values; the fix does not claim to rewrite history.”

Key code segments and file locations

Ignore environment files

File: demo/secure/pubudu-pos-front-end-secure/.gitignore Lines: 26-31

```
.env
.env.*
!.env.example
```

CI rejects tracked environment files

File: demo/secure/pubudu-pos-front-end-secure/.github/workflows/security.yml Lines: 86-105

```
if git ls-files --error-unmatch .env; then
  echo 'Environment files are tracked'; exit 1
fi
```

V-16 - Sinhala guide first, English speech underneath

Sinhala guide - මේ කොටස කරන විදිහ

Video steps

Original browser DevTools එකේ localStorage token/user role පෙන්වන්න; personal token value එක blur/mask කරන්න. Secure GET /me, memory/session handling, logout cleanup සහ API authorization code පෙන්වන්න. Screenshot names: V16-01-original-storage-role.png, V16-02-secure-server-role.png, V16-03-session-code-test.png.

V-16 - browser token සහ role trust

Original `src/lib/auth.js` token සහ full user record `localStorage` තුළ තැබුවා. `localStorage` tab/browser close කළත් ඉතිරි වේ; එම origin එකම JavaScript එකකට කියවිය හැක. Role එකත් client storage එකත් බලන නිසා admin UI එක ව්‍යාජව පෙන්විය හැකි වුණා. UI එක admin වුණත් API admin middleware block කළ යුතුයි; ඒ backend control Ravindu workstream එකෙන් එක් වුණා.

දැනට token එක module memory variable එකම, refresh සඳහා per-tab `sessionStorage` mirror එකම ඇත. User cache display සඳහා පමණයි. `src/contexts/AuthContext.jsx` app boot එකම GET `/me` ගෙන serverගේ role/status භාවිතා කරයි; fail වුණොත් cached userට fall back නොවෙයි. `src/lib/api.js` bearer token requestට එකතු කර 401 පසු cleanup කරයි. Logout එක old token/user keys චිතරක් නොව invoice/GRN drafts වැනි business data keys ද clear කරයි.

වැදගත් limitation: `sessionStorage` ද same-origin JavaScript එකට කියවිය හැක. එය httpOnly cookie එකක් වගම XSS-proof නොවෙයි. මනේ වාසිය persistence කවේ වීම හා old `localStorage` exposure අඩු වීම; අර්ධාන XSS fix එක HTML escaping. NF - 2 සඳහා future httpOnly cookie architecture අවශ්‍යයි.

Flow එක මතක තබාගන්න

```
Page load → session token ■■■■■?
■■■■■ → login screen
■■■■■ → GET /api/me
200 + active user → fresh server role ■■■■ UI
401 / fail → token, user cache, drafts ■■■■ ■■ login
```

English speech - read this in the video

V-16 - Client-trusted role and browser token exposure

[Show the Original browser storage and the client-side role check.]

"The Original frontend stored the bearer token and the full user record in local storage. It also trusted the stored role to decide which administrator interface to display. JavaScript running in the same origin could read that token, and changing the stored role could make the administrator UI appear even though the server had not granted that role."

[Show the Secure GET `/me` flow, token accessors and cleanup.]

"The Secure frontend asks the server for the current user and uses the server response as the source of truth for role and active status. Token persistence is reduced and centralized, failed authentication removes the session and business drafts, and dangerous direct local-storage access is blocked by the security lint rules. The API still enforces authorization, so changing browser storage cannot grant permission."

Key code segments and file locations

Server is the source of truth for identity

File: `demo/secure/pubudu-pos-front-end-secure/src/contexts/AuthContext.jsx` Lines: 188-208

```
const response = await api.get('/me');
setUser(response.data.user);
// Do not trust a cached client-side role.
```

Centralized token cleanup

File: `demo/secure/pubudu-pos-front-end-secure/src/lib/api.js` Lines: 20-56

```
if (response.status === 401) {
  auth.clear();
  sessionStorage.removeItem('user');
}
```

V-17 - Sinhala guide first, English speech underneath

Sinhala guide - මමේ කතාටස කරන විදිහ

Video steps

Original upload page එක සහ extension-only validation එක පෙන්වන්න. Secure validateUpload.js හි size, extension, MIME, magic bytes සහ row-limit flow එක පෙන්වන්න. Real malicious file upload නොකර test fixture/code භාවිතා කරන්න. Screenshot names: V17-01-original-upload-check.png, V17-02-secure-upload-rejected.png, V17-03-upload-validation-test.png.

V-17 - spreadsheet/CSV upload validation

Original screens කිහිපයක් file එක SheetJS parser වනෙ යවන විට එක සමාන validation කර නැහැ. <input accept=".xlsx"> කියන්නේ file picker hint එකක්; file එක rename/drag-drop කළොත් bypass කළ හැක. Parser එකමේ පැරණි advisories ද තිබුණා. src/lib/validateUpload.js එක්කර Products.jsx, Services.jsx, Stock/GrnAdjust.jsx, GRNForm/CSVImportModal.jsx වලින් යොදා ඇත.

Validation order එක: file නිමේද emptyද 10 MB ඉක්මවයිද extension .xlsx/.xls/.csvද browser MIME type plausibleද මුල් bytes/magic signature හරිද parse පසු 5000 rows ඉක්මවයිද. .xlsx සඳහා ZIP PK bytes; .xls සඳහා OLE2 හමෝ ZIP; .csv සඳහා binary NUL check. MAX_UPLOAD_BYTES හා MAX_IMPORT_ROWS constants source එකමේ බලන්න.

```
const head = new Uint8Array(await file.slice(0, 8).arrayBuffer());
if (extension === ".xlsx" && !startsWith(head, ZIP_MAGIC)) {
  return { ok: false, error: "That file is not a valid .xlsx workbook." };
}
```

මගේ logic එකමේ සරල කතාටසකි. Magic bytes valid වුණා කියා file එක මුද්‍රමනින් safe නොවමේ; parser upgrade, row limit සහ output escaping ද අවශ්‍යයි. Uploaded product names receipt XSS path එකට යන්න පුළුවන් නිසා V-17 සහ V-13 එකට සමීක්‍රමයි.

English speech - read this in the video

V-17 - Weak spreadsheet and CSV upload validation

[Show the Original upload input and the old validation path.]

"The Original upload relied mainly on the file-picker accept attribute and did not apply one consistent validation path to every import screen. File extensions and browser MIME values can be changed by an attacker, so they are not sufficient security controls."

[Show validateUpload.js and the upload tests.]

"The Secure upload helper validates presence, file size, extension, plausible MIME type, magic bytes and the maximum number of parsed rows. XLSX files are checked for the ZIP signature, legacy XLS files for the expected container and CSV files for unsafe binary content. Parser updates and output escaping are still required as defence in depth."

Key code segments and file locations

Upload size, signature and extension checks

File: demo/secure/pubudu-pos-front-end-secure/src/lib/validateUpload.js Lines: 35-39 and 75-185

```
if (file.size > MAX_UPLOAD_BYTES) return invalid('File is too large.');
```

```
if (!allowedExtensions.has(extension)) return invalid('Unsupported type.');
```

```
if (!hasExpectedMagicBytes(file, extension)) return invalid('Invalid file signature.');
```

V-20 - Sinhala guide first, English speech underneath

Sinhala guide - මමේ කතෘටස කරන විදිහ

Video steps

Original cashier account එකෙන් stock/GRN page open කර browser Network response එකේ `actual_cost`, `stock_price`, `margin` වගේ fields පෙන්වන්න. Secure employee response එකේ ඒ fields නැති බවත් admin response එකේ permitted බවත් පෙන්වන්න. Screenshot names: `V20-01-original-cost-leak.png`, `V20-02-secure-filtered-response.png`, `V20-03-response-filtering-code-test.png`.

V-20 - cashierට wholesale cost/margin leak වීම (backend part)

Frontend screen එක cost column එක සඟවා තිබුණත් `/api/grns`, `/api/grns/stock`, `/api/grns/product/{code}/batches`, `/api/grns/{id}`, `/api/grns/code/{grnCode}` responses තුළ `stock_price`, `actual_cost`, `discounts`, `total_cost`, `total_profit` වැනි supplier/financial values තිබුණා. Cashierට stock හා selling price අවශ්‍යයි; supplier cost/margin අවශ්‍ය නැහැ. DevTools Network response එකෙන් data කියවිය හැකි නිසා frontend hide කිරීම හොඳ නැහැ. මෙය object property-level authorization ගැටලුවකි.

`app/Models/Concerns/HidesCostFromEmployees.php` trait එක Grn සහ GrnItem model serialization එකේ cost fields default hidden කරයි. Admin viewer කෙනෙකුට `revealCostTo($request->user())` මගින් විශේෂයෙන් පෙන්වයි. User නැත්නම් fail closed. Controller අතින් array එකක් හදන paths වල model \$hidden කිරීම නොකරන නිසා `costFieldsVisibleTo()` check එකෙන් keys omit කරයි.

```
public static function costFieldsVisibleTo(?User $user): bool
{
    return $user !== null && $user->role === 'admin';
}
```

`app/Http/Controllers/Api/GrnController.php`, `app/Models/Grn.php`, `app/Models/GrnItem.php` සහ trait එක එකට බලන්න. Employeeට stock lookup 200 හා `selling_price` තව ලැබේ; cost fields නොලැබේ. Adminට cost fields ලැබේ. `tests/Feature/Security/ResponseFilteringTest.php` හි endpoint five, model default, null user, admin behaviour ඇතුළු 16 tests. `evidence/before/v20-cost-leak.txt` original reproduction; `evidence/after/poc-dynamic-testing.txt` fix proof.

English speech - read this in the video

V-20 - Wholesale cost and margin leaked to employees

[Show the Original browser Network response for a stock or GRN endpoint.]

"In the Original API response, a cashier could see supplier cost, stock price, discounts and profit-related fields even when the screen hid those columns. Hiding a column with CSS is not authorization because the complete JSON response is visible in the Network panel."

[Show the Secure model trait, controller filtering and response tests.]

"The Secure backend hides cost and margin fields by default and reveals them only when the authenticated user is an administrator. Controller paths that build arrays manually apply the same role decision. An employee still receives permitted stock quantity and selling price, but not wholesale cost or margin. The response-filtering tests cover all protected endpoints and both employee and administrator behavior."

Key code segments and file locations

Hide cost fields by default

File: demo/secure/pubudu-pos-api-secure/app/Models/Concerns/HidesCostFromEmployees.php Lines: 94-128

```
public static function costFieldsVisibleTo(?User $user): bool
{
    return $user !== null && $user->role === 'admin';
}
```

Response filtering is tested for both roles

File: demo/secure/pubudu-pos-api-secure/tests/Feature/Security/ResponseFilteringTest.php Lines: 63-108

```
$this->actingAs($employee)
->getJson('/api/grns/stock')
->assertJsonMissingPath('data.0.actual_cost');
```

Conclusion

Sinhala conclusion

මමේ memberගේ assigned work එකේදී Original bug එක, Secure fix එක, code evidence එක සහ test result එක පැහැදිලිව පෙන්වන්න. තමන් දැක්වීමට කළ contribution එකට පමණක් claim කරන්න. Team-level test counts තමන්ගේ individual count ලෙස කියන්න එපා.

English conclusion - say this

"To conclude, I compared the Original behavior with the Secure behavior, explained the impact of each finding, and connected each fix to source code and automated test evidence. The controls are enforced at the correct boundary, and the remaining limitations are documented rather than hidden. Thank you."

Recording safety

- Passwords, bearer tokens, client secrets and personal Google accounts must stay hidden.
- Use code and test evidence for destructive or unsafe attack steps.
- Keep the cursor on the exact result or code line being discussed.